

Ejercicio No. 1 (25%) – Para el siguiente programa:

```

void function (int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 1; j+n/2 <= n; j++) {
            for (k = 1; k <= n; k = k*2) {
                counter++;
            }
        }
    }
}

```

a) Encuentre la complejidad de tiempo en notación Big-Oh. Deje todo su procedimiento.

Instrucción	Operación	Costo
counter = 0	Asignación	C_1
for i	ciclo	A veces
for j	ciclo	B veces
for k	ciclo	L veces
counter++	suma y asignación	$C_1 + C_3$

Iteraciones

iteraciones de i

$$i \in \{ \lfloor \frac{n}{2} \rfloor, \lfloor \frac{n}{2} \rfloor + 1, \dots, n \}$$

$$A(n) = n - \lfloor \frac{n}{2} \rfloor + 1$$

$$= \lceil \frac{n}{2} \rceil + 1$$

iteraciones de j

$$j + \lfloor \frac{n}{2} \rfloor \leq n$$

$$j \leq n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$$

$$B(n) = \lceil \frac{n}{2} \rceil$$

iteraciones de k

$$K_t = 2^t, \quad t = 0, 1, 2, \dots$$

$$2^t \leq n \Leftrightarrow t \leq \log_2 n$$

$$t \in \{0, 1, \dots, \lfloor \log_2 n \rfloor\}$$

$$L(n) = \lfloor \log_2 n \rfloor + 1$$

$$L(n) \lfloor \log_2 n \rfloor + 1$$

Los ciclos no son dependientes

Tiempo de ejecución

$$T(n) = C_1 + A(B(L(C_1 + C_3)))$$

$$= C_1 + ABL(C_1 + C_3)$$

$$A \approx \frac{n}{2}, B \approx \frac{n}{2}, L \approx \log_2 n$$

$$\approx C_1 + \frac{n}{2} \cdot \frac{n}{2} \log_2 n (C_1 + C_3)$$

$$= \underbrace{\frac{C_1 + C_3}{4}}_a n^2 \log_2 n + \underbrace{C_1}_b$$

límite Big O

$$\lim_{n \rightarrow \infty} \left| \frac{T(n)}{n^2 \log_2 n} \right| = \lim_{n \rightarrow \infty} \left| a + \frac{b}{n^2 \log_2 n} \right|$$

$$= a = \frac{C_1 + C_3}{4} \leq C$$

$$\underline{T(n) = O(n^2 \log n)}$$

Ejercicio No. 2 (25%) – Para el siguiente programa:

```
void function (int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf ("Sequence\n");
            break;
        }
    }
}
```

a) Encuentre la complejidad de tiempo en notación Big-Oh. Deje todo su procedimiento.

Instrucción	Operación	Costo
$n \leq 1$	Comparación	C_2
return	Retorno	C_6
for i	Ciclo	n veces
for j	Ciclo	1 vez
printf	Impresión	C_4
break	Salir del ciclo	$C_{1.5}$

Print break	im presión salir del ciclo	C_4 C_5
----------------	-------------------------------	----------------

Iteraciones ($n \geq 2$)

$i = 1, \dots, n \Rightarrow n$ iteraciones
 $j = 1$ iteración cada i

Tiempo de ejecución

$$n \leq 1 \Rightarrow T(n) = C_2 + C_6$$

Tiempo Constante

$$n \geq 2: T(n) = C_2 + n(C_4 + C_5)$$

$$= \underbrace{(C_4 + C_5)}_a n + \underbrace{C_2}_b$$

límite Big O

$$\lim_{n \rightarrow \infty} \left| \frac{T(n)}{n} \right| = \lim_{n \rightarrow \infty} \left| a + \frac{b}{n} \right|$$

$$= a = C_4 + C_5 \leq C$$

$$\underline{T(n) = O(n)}$$

Ejercicio No. 3 (25%) – Para el siguiente programa:

```
void function (int n) {
    int i, j;
    for (i=1; i<=n/3; i++) {
        for (j=1; j<=n; j+=4) {
            printf("Sequence\n");
        }
    }
}
```

a) Encuentre la complejidad de tiempo en notación Big-Oh. Deje todo su procedimiento.

Operaciones

instrucción	Costo
for i	P veces
for j	Q veces
print	C_4

Iteraciones

$$i = 1, \dots, \lfloor n/3 \rfloor \Rightarrow P = \lfloor \frac{n}{3} \rfloor$$

$$j = 1, 5, 9, \dots, 1 + 4t \leq n \Rightarrow t \leq \frac{n-1}{4}$$

$$j = 1, 5, 9, \dots, 1 + 4t \leq n \Rightarrow t \leq \frac{n-1}{4}$$

$$\Rightarrow Q = \left\lfloor \frac{n-1}{4} \right\rfloor + 1 = \left\lceil \frac{n}{4} \right\rceil$$

Tiempo de ejecución

$$T(n) = P(QC_4) = PQC_4$$

$$P \approx \frac{n}{3}, Q \approx \frac{n}{4}$$

$$\approx \frac{n}{3} \cdot \frac{n}{4} \cdot C_4$$

$$= \frac{C_4}{12} n^2$$

límite Big O

$$\lim_{n \rightarrow \infty} \left| \frac{T(n)}{n^2} \right| = a = \frac{C_4}{12} \leq C$$

$$\underline{T(n) = O(n^2)}$$

Ejercicio No. 4 (10%) – Encuentre el mejor caso, caso promedio y peor caso del algoritmo de Búsqueda Lineal (Linear Search). Deje todo su procedimiento.

Algoritmo de búsqueda lineal:

```
int linearSearch(int a[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (a[i] == x)
            return i;
    }
    return -1;
}
```

Sea k la posición (contando desde 1) donde está x, es decir, x se encuentra en a[k-1].

Operaciones

Instrucción	Costo
for i	n veces
a[i] = x	C ₂
return	C ₆

$$x \text{ en } R: T_R = RC_2 + C_6$$

$$x \text{ no está: } T_{no} = nC_2 + C_6$$

Mejor caso

$$T_m(n) = (T_1) = C_2 + C_6 \leftarrow x \text{ en } [0]$$

$$\lim_{n \rightarrow \infty} \left| \frac{T(n)}{n} \right| = c_2 + c_6 \leq C$$

$$\underline{T(n) = O(n)}$$

Peor caso

$$T_w(n) = T_n = \underbrace{c_2 n}_a + \underbrace{c_6}_b$$

$$\lim_{n \rightarrow \infty} \left| \frac{T_w(n)}{n} \right| = \lim_{n \rightarrow \infty} \left| a + \frac{b}{n} \right| = c_2 \leq C$$

$$\underline{T_w(n) = O(n)}$$

Caso promedio

$$T_p(n) = \frac{1}{n+1} \left[\sum_{x=1}^n (x c_2 + c_6) + (n c_2 + c_6) \right]$$

$$= \frac{1}{n+1} \left[c_2 \frac{n(n+1)}{2} + n c_6 + n c_2 + c_6 \right]$$

$$= \frac{c_2}{2} n + c_6 + \frac{c_2}{n+1}$$

$$\approx \underbrace{\frac{c_2}{2} n}_a + \underbrace{(c_6 + \frac{c_2}{n+1})}_b \quad \frac{n}{n+1} \rightarrow 1$$

$$\lim_{n \rightarrow \infty} \left| \frac{T_{prom}(n)}{n} \right| = \lim_{n \rightarrow \infty} \left| a + \frac{b}{n} \right| = \frac{c_2}{2} \leq C$$

$$\underline{T_{prom}(n) = O(n)}$$

Ejercicio No. 5 (15%) – Decida si los siguientes enunciados son verdaderos o falsos. Debe justificar sus respuestas para recibir los créditos completos.

f , g y h representan tiempos de ejecución, así que son positivas para n grande y se pueden quitar los valores absolutos de las definiciones.

a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$.

Hipótesis def de Θ

$$f = \Theta(g) \Rightarrow \exists a_1, a_2, r_1 > 0 : a_1 g(n) \leq f(n) \leq a_2 g(n) \quad \forall n \geq r_1$$

$$g = \Theta(h) \Rightarrow \exists b_1, b_2, r_2 > 0 : b_1 h(n) \leq g(n) \leq b_2 h(n) \quad \forall n \geq r_2$$

Sea $r = \max(r_1, r_2)$ para $n \geq r$:

$$f(n) \geq a_1 g(n) \geq a_1 b_1 h(n) \Rightarrow h(n) \leq \frac{1}{a_1 b_1} f(n)$$

$$f(n) \leq a_2 g(n) \leq a_2 b_2 h(n) \Rightarrow h(n) \geq \frac{1}{a_2 b_2} f(n)$$

$$\underbrace{\frac{1}{a_2 b_2}}_{c_1 > 0} f(n) \leq h(n) \leq \underbrace{\frac{1}{a_1 b_1}}_{c_2 > 0} f(n) \quad \forall n \geq r$$

$$\underline{h = \Omega(f) \Rightarrow \text{verdadero}}$$

Se usan la transitividad ($f = O(h)$) y la Propiedad $f = O(g) \Leftrightarrow g = O(f)$

b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$.

Hipótesis (def de O)

$$f = O(g) \Rightarrow \exists a, r_1 > 0 : f(n) \leq a g(n) \quad \forall n \geq r_1$$

$$g = O(h) \Rightarrow \exists b, r_2 > 0 : g(n) \leq b h(n) \quad \forall n \geq r_2$$

Sea $r = \max(r_1, r_2)$. Para $n \geq r$:

$$f(n) \leq a g(n) \leq a b h(n)$$

$$h(n) \geq \underbrace{\frac{1}{ab}}_{c > 0} f(n)$$

$$\underline{h = \Omega(f) \Rightarrow \text{Verdadero}}$$

c) $f(n) = \Theta(n^2)$, donde $f(n)$ está definido por ser el tiempo de ejecución del programa de Python A(n):

```
def A(n):
    atupla = tuple(range(0, n)) # una tupla es una versión inmutable de una
                                # lista, que puede ser hashada
    S = set()
    for i in range(0, n):
        for j in range(i + 1, n):
            S.add(atupla[i:j]) # añade la tupla (i,...,j-1) al set S
```

Operaciones

$$\text{tuple(range(0, n))} : O(n)$$

$$\text{atupla}[i:j] : O(l)$$

$$\text{S.add}(\dots) : O(1)$$

$$\text{costo}(i, j) = a l + b = a(j - i) + b$$

Número de iteraciones del ciclo

$$n-1 \quad n-1 \quad \dots \quad n-1 \quad , \quad \dots \quad \underline{n-1}$$

Número de iteraciones del ciclo

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-1} (n-1-i) = \sum_{m=0}^{n-1} m$$

$m = n-1-i$

$$= \frac{n(n-1)}{2}$$

Tiempo de ejecución

$$f(n) = C_1 n + \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} [a(j-i) + b] + C_0$$

Suma de longitudes

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (j-i) = \sum_{i=0}^{n-1} \sum_{l=1}^{n-1-i} l$$

$l = j-i$

$$= \sum_{i=0}^{n-1} \frac{(n-1-i)(n-i)}{2}$$
$$= \sum_{m=0}^{n-1} \frac{m(m+1)}{2}$$

$m = n-1-i$

$$= \frac{1}{2} \left[\sum_{m=0}^{n-1} m^2 + \sum_{m=0}^{n-1} m \right]$$
$$= \frac{1}{2} \left[\frac{(n-1)n(2n-1)}{6} + \frac{(n-1)n}{2} \right]$$
$$= \frac{n(n-1) [(2n-1) + 3]}{12}$$
$$= \frac{n(n-1)(n+1)}{6} = \frac{n^3 - n}{6}$$

$$\Rightarrow f(n) = \frac{a}{6}(n^3 - n) + \frac{b}{2}n(n-1) + C_1 n + C_0$$

Límites

$$\lim_{n \rightarrow \infty} \left| \frac{f(n)}{n^2} \right| = \lim_{n \rightarrow \infty} \left| \frac{a}{6} \left(n - \frac{1}{n} \right) + \frac{b}{2} \left(1 - \frac{1}{n} \right) + \frac{C_1}{n} + \frac{C_0}{n^2} \right| = \infty$$

$$\Rightarrow \nexists C_2 \text{ con } \lim_{n \rightarrow \infty} \left| \frac{f(n)}{n^2} \right| \leq C_2 \Rightarrow f \neq \theta(n^2)$$

$$\lim_{n \rightarrow \infty} \left| \frac{f(n)}{n^3} \right| = \frac{a}{6} \Rightarrow f = \theta(n^3)$$

$$\underline{f(n) = \theta(n^3) \neq \theta(n^2) \Rightarrow \text{false}}$$